Transformer_Architecture_notes

1. Introduction:

The Transformer is a deep learning architecture. It revolutionized natural language processing (NLP) by relying entirely on attention mechanisms, eliminating the need for recurrent networks.

Key Features

- Processes all input tokens in parallel.
- Captures long-range dependencies effectively.
- Faster training than recurrent models.
- Scales well to very large datasets.

2. Overall Architecture:

A Transformer consists of two main parts:

- Encoder
- Decoder

"Input → Encoder → Encoder Output → Decoder → Output"

Encoder:

- Reads and understands the input sequence.
- Produces contextual representations.

Decoder:

- Generates the output sequence.
- Uses encoder outputs and previously generated tokens.

3. Main Components

A. Input Embedding:

Words are converted into dense numerical vectors called embeddings.

B. Positional Encoding:

Since Transformers process all words simultaneously, they need information about word order. Positional encoding is added to embeddings.

Purpose:

- Preserves sequence order.
- Helps distinguish word positions.

C. Self-Attention:

Self-attention allows every word to focus on other relevant words in the sentence.

Ex: "The animal didn't cross the street because it was tired."

The word it attends strongly to “animal.”

4. Query, Key, and Value (Q, K, V):

Each input vector is transformed into:

- Query (Q)
- Key (K)
- Value (V)

5. Multi-Head Attention:

Instead of using one attention mechanism, Transformers use several attention heads simultaneously.

Advantages:

- Learns different relationships.
- Improves representation quality.
- Captures syntax and semantics together.

6. Feed Forward Network (FFN):

Each encoder and decoder layer contains a fully connected neural network.

Purpose:

- Learns complex feature transformations.
- Applied independently to each token.

7. Residual Connection

Each sub-layer uses:

Output = Layer(x) + x

Benefits:

- Easier optimization.
- Better gradient flow.
- Enables very deep networks.

8. Layer Normalization

Applied after residual connections.

Purpose:

- Stabilizes training.
- Speeds convergence.
- Reduces internal covariate shift.

9. Encoder Layer:

Each encoder block contains:

Input

↓

Multi-Head Attention

↓

Add & Normalize

↓

Feed Forward Network

↓

Add & Normalize

↓

Output

Typical Transformers stack multiple encoder layers.

10. Decoder Layer:

Each decoder block contains:

Masked Multi-Head Attention

↓

Add & Normalize

↓

Encoder-Decoder Attention

↓

Add & Normalize

↓

Feed Forward Network

↓

Add & Normalize

11. Masked Attention:

The decoder cannot see future words during training.

The model cannot look ahead to later words.

Purpose:

- Prevents information leakage.
- Enables autoregressive generation.

12. Encoder–Decoder Attention:

The decoder attends to encoder outputs. The decoder focuses on relevant encoder representations while generating each output token.

13. Training Process:

1. Input sentence
2. Tokenization
3. Embedding
4. Positional encoding
5. Encoder processing
6. Decoder processing
7. Prediction
8. Loss computation
9. Backpropagation
10. Weight update

14. Advantages:

- Parallel computation.
- Handles long-range dependencies effectively.
- Highly scalable.
- State-of-the-art performance in many NLP tasks.

- Foundation for modern large language models.

15. Limitations:

- Requires large datasets for best performance.
- Computationally expensive for very long inputs.

16. Applications:

- Machine translation
- Text summarization
- Question answering
- Text generation
- Chatbots
- Sentiment analysis
- Speech recognition